

Authentication Middleware Changes

Overview

Updated authentication middleware to support both OAuth Bearer tokens and API key headers, with OAuth taking precedence.

Changed File

`app/middleware/auth.py` - `APIKeyValidationMiddleware` class

Commit

`0ce3ae6` - "fix: add OAuth Bearer token support to auth middleware"

Changes

Before

```
async def __call__(self, request: Request, call_next):
    api_key = request.headers.get("X-API-Key")
    if not api_key:
        return JSONResponse(
            status_code=401,
            content={"detail": "X-API-Key header required"},
        )

    # Validate API key...
    response = await call_next(request)
    return response
```

Limitations:

- Only supported API key authentication
- Could not use Cloud Run's native IAM-based authentication
- Incompatible with OAuth tokens

After

```
async def __call__(self, request: Request, call_next):
    # Check for OAuth Bearer token first
    auth_header = request.headers.get("Authorization", "")
    if auth_header.startswith("Bearer "):
        # OAuth token provided, allow request
        logger.info(f"Request {request_id} authenticated via OAuth Bearer token")
```

```

        response = await call_next(request)
        return response

    # Fall back to API key validation
    api_key = request.headers.get("X-API-Key")
    if not api_key:
        return JSONResponse(
            status_code=401,
            content={"detail": "Authorization header or X-API-Key header
required"},
        )

    # Validate API key...
    response = await call_next(request)
    return response

```

Improvements:

- Supports OAuth Bearer tokens via `Authorization: Bearer <token>`
- Maintains backward compatibility with `X-API-Key` header
- OAuth takes precedence (checked first)
- Updated error messages to reflect both auth methods
- Logs which authentication method was used

Authentication Methods

1. OAuth Bearer Token (Cloud Run)

```

# Generate token
AUTH_TOKEN=$(gcloud auth print-identity-token --audiences https://your-service-
url)

# Use in request
curl -H "Authorization: Bearer $AUTH_TOKEN" https://your-service-url/chat

```

Benefits:

- Uses Cloud Run's IAM security
- No separate API key management
- Automatic token refresh
- Supports user identity authentication

2. API Key Header (Legacy)

```

# Set API key
curl -H "X-API-Key: your-api-key" http://localhost:8080/chat

```

Benefits:

- Simple for local development
- Useful for programmatic access
- Can be scoped per application

Testing

Local Development with API Key

```
export ISPILOT_API_KEY="test-key-123"
python -m uvicorn app.main:app --reload --port 8080

# Test
curl -X POST http://localhost:8080/chat \
  -H "X-API-Key: test-key-123" \
  -H "Content-Type: application/json" \
  -d '{"user_id": "test", "message": "hello"}'
```

Cloud Run with OAuth

```
# Deploy
bash deploy.sh

# Generate token
AUTH_TOKEN=$(gcloud auth print-identity-token --audiences https://ispilot-api-46y2f3tyja-uc.a.run.app)

# Test
curl -X POST https://ispilot-api-46y2f3tyja-uc.a.run.app/chat \
  -H "Authorization: Bearer $AUTH_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"user_id": "test", "message": "hello"}'
```

Priority Order

The middleware checks authentication in this order:

1. **Authorization Bearer header** → If present and valid format, accept request
2. **X-API-Key header** → If Bearer token not present, validate API key
3. **Reject** → If neither provided, return 401 Unauthorized

Error Messages

Scenario	Status	Message
No auth header	401	"Authorization header or X-API-Key header required"

Scenario	Status	Message
Invalid API key	401	"Invalid API key"
Bearer token present	200	Request accepted (token validation handled by Cloud Run)

Backward Compatibility

☑ **Fully backward compatible** - Existing code using `X-API-Key` header continues to work without changes.

Clients can migrate to OAuth at their own pace:

1. Keep using `X-API-Key` during transition
2. New code can use `Authorization: Bearer`
3. Both methods work simultaneously

Security Implications

OAuth Bearer Tokens

- ☑ Leverages Cloud Run's IAM security
- ☑ No secrets stored in environment
- ☑ Automatic token expiration
- ☑ Identity-based access

API Key Headers

- ⚠ Still supported for backward compatibility
- ⚠ Secrets must be managed carefully
- ⚠ No automatic expiration
- ⚠ Key-based access (not identity-based)

Recommendation: Use OAuth for production, API key for local development only.

Related Changes

- Session Service Firestore Fallback: [1881c51](#)
- Deploy Script Fixes: [5dcd6c3](#), [f5cfeae](#)
- See [DEPLOYMENT_SUMMARY_2026-08-28.md](#)